Exercise 3 AXI and Memory Interfaces

Objectives

The goal of this exercise is to implement a system in which data received from UART is stored in FPGA's BRAM or external DDR2 memory. AXILite interface will be used to communicate with BRAM controller or the Memory Interface Generator (MIG), which in turn reads and writes data to/from DDR RAM. This will allow storing of larger data sets, like images. BRAM setup will be used for fast testing scenario, whereas MIG + DDR2 will be the final target system.

The exercise is divided into three parts:

Part 1 focuses on AXI Stream to Lite interface and BRAM

Part 2 implements advanced testbench which can read data and commands from files

Part 3 implements complete system with DDR2 memory and advanced testbench

Part 1: AXI and BRAM Memory

Using design from Exercise 2, provided VHDL files (`top.vhd`, `fpga_top_BRAM.vhd`, `tb_fpga_top_BRAM.vhd`), and information from Memory tutorial, create a design which is capable of the following:

1. Reading 4 bytes of data over UART and writing them to BRAM memory over AXI Lite interface to 4 different memory locations (each byte will be individually read over UART and right after that written to BRAM)

2. Reading the 4 bytes of data from BRAM memory and sending them back over UART (each byte will be individually read from BRAM and right after that written to UART)

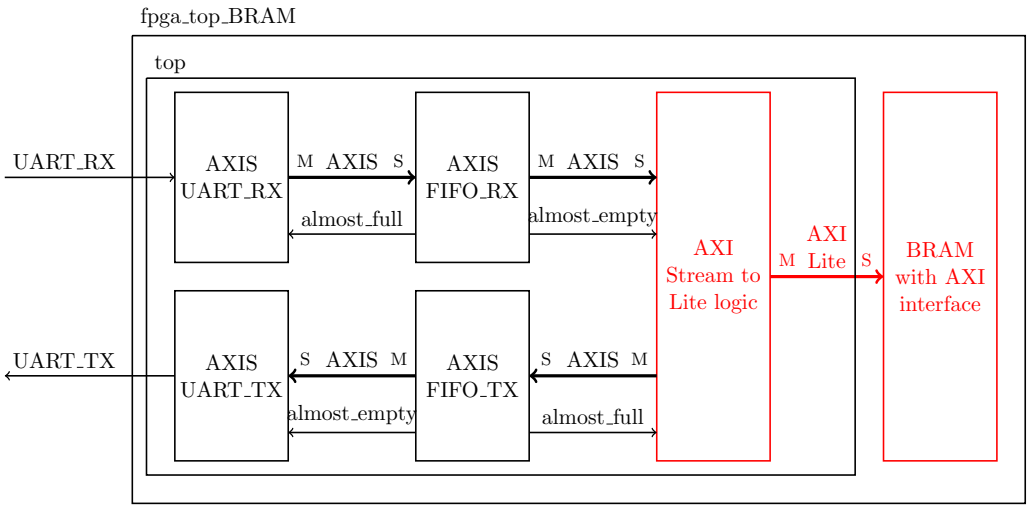


Figure 3.1: Block diagram of a system with BRAM memory (red blocks are new)

Two FIFOs from the previous design will need to be configured for the purpose of this exercise. The transmit FIFO (to the PC) should be 1024 bytes large and the receive FIFO 512 bytes. You

will need to generate two separate FIFO IP cores such that they can be configured with these two different sizes.

Logic will need to be written for converting the AXI Stream signals to AXI Lite signals. In particular, AXI Stream does not have an address, so the new logic should handle address generation for each received byte of data. The first address should be 0 and the next addresses would be incremented by 4.

This design should be placed in the `top` entity, and an instantiation of this entity can be found in `fpga_top_BRAM.vhd`. Use this testbench, along with BRAM IP component, from the Memory tutorial to provide complete setup simulation.

Your testbench (`tb_fpga_top_BRAM`) needs to mimic desired behavior and be capable of sending and receiving 4 bytes of data.

A high level diagram of this system is presented in figure 3.1.

```
tb_fpga_top_BRAM
└── fpga_top_BRAM
    ├── top
    └── BRAM
```

Much of this design is already given in the Memory and AXI tutorials, however, it is important to pay attention to the required reset values when creating the design. A high level diagram, showing the blocks (memory, AXI buses, AXI controllers, UART, etc.) must be drawn.

Use MATLAB or screen to test the design in hardware.

Part 1 needs to be signed off by 10/23/2026.

Part 2: Testbench with commands.

In order to automate testing of the design and make changes to the input data more easily, an advanced testbench will be written. The testbench must read commands from a file using the `readline` and `read` commands. The data sent to the FPGA must be transmitted in binary rather than ASCII.

The testbench should be capable of parsing a command file and executing the commands it finds there. The commands are:

- Test Memory: Send one or more hex values specified after the command, then read the bytes back over UART and verify that they are the same.
- Wait: Wait for X cycles of the clock.

The strings for these commands are `TEST` and `WAIT`. In addition to processing these commands, the testbench should log all received data to an output text file.

The "Test Memory" command should be able to accept an arbitrary number of hexadecimal arguments:

`TEST <XX> <XX> <XX>...<XX>`

There is no need to convert the hex data to ASCII, it should simply be sent as is.

The "Wait" command should accept a single argument, and wait for that many cycles of the 100 MHz clock:

`WAIT <num_cycles>`

It may be helpful to search for the VHDL function `hread()` when implementing these commands, in particular, the version of the function that accepts three arguments.

To make adequate use of the testbench, modify the design to accept a configurable number of writes and reads. For this exercise, this number may be hardcoded to four, but in later exercises, the exact amount will be not be determined until run time. Increment the address by 4 between writes. After performing the four writes, the design should then read the four bytes back from memory and return them over UART. The testbench should perform this operation twice, each with different hex values, and wait for 100 clock cycles between tests.

Initial functionality should be developed using the `tb_fpga_top_BRAM` module.

Part 3: MIG and DDR2 memory.

The final setup for this exercise is presented in figure 3.2. The `top` design will be taken from **Part 1** of this exercise and textbench from **Part 2**.

AXI Protocol Converter and MIG controller are part of `fpga_top_DDR2` design. Steps to create an MIG component are described in Memory tutorial. DDR2 memory model is also provided.

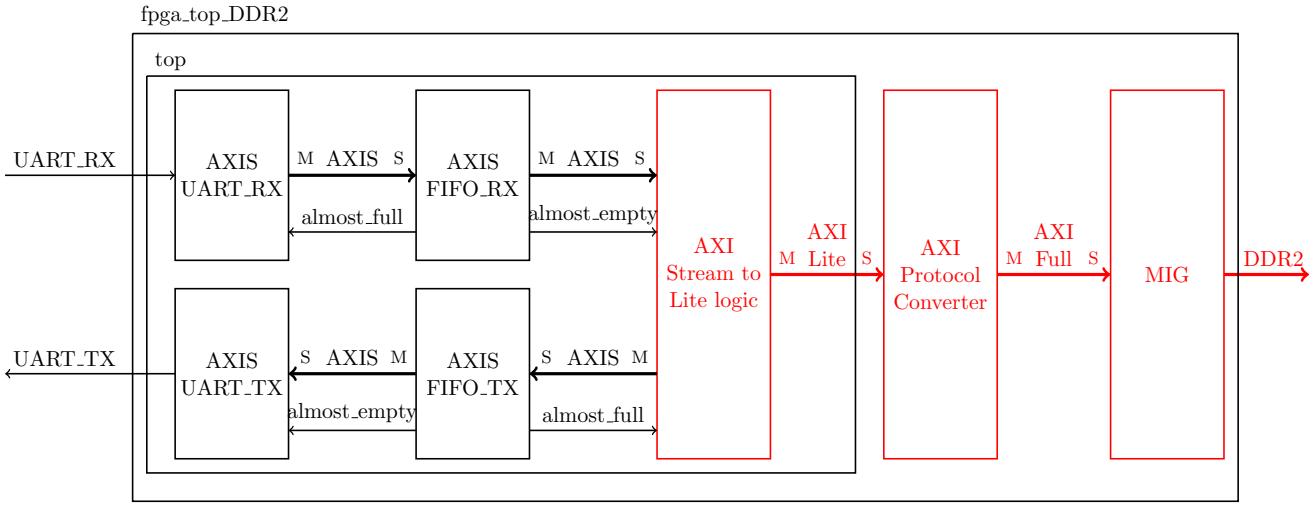


Figure 3.2: Block diagram of a system with DDR2 memory (red blocks are new)

Perform this setup tests using `tb_fpga_top_DDR2`. Your testbench needs to use flexible approach developed in **Part 2** of this exercise. A high level diagram of this system is presented in figure 3.1.

Because this simulation can take a long time, it is helpful to log all of the waves. This can be done by executing `log_wave -r /` in the TCL console before pressing run, but after restarting the simulation. If a wave was originally not added, simply add it to the simulation window, and the entire waveform will appear.

Use MATLAB or screen to test the design in hardware.

Extra credit: 20% extra credit for the entire lab will be given for implementing an AXI Stream to AXI Full interface (without the need for AXI Protocol Converter) that sends/receives data in bursts of 4 bytes.

Exercise 3: AXI and Memory Interfaces

Student's Name:_____ Section:_____

Demo		Point Value	Points Earned	Date
Part 1: AXI/BRAM	Simulation	15		
	RTL + Hardware demo	15		
Part 2: Testbench	Simulation	20		
Part 3: DDR2	Simulation	15		
	RTL + Hardware Demo	15		

To receive any grading credit students must earn points for both the demonstration and the report.

Report		Point Value	Points Earned	Comments
Report	Discussion	20		
Total for demo and report		100		

An electronic submission of project files via MyCourses is required. For information on how to properly submit the files, see the document "MyCoursesTut.pdf" on MyCourses.

Required file structure of tarball submission for this exercise:

```
Ex3
├── Part1
│   ├── [all necessary .vhd files]
│   └── Part1.xdc
├── Part2
│   └── [all necessary .vhd files]
└── Part3
    ├── [all necessary .vhd files]
    └── Part3.xdc
```